

**LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 17**



**DISUSUN OLEH:
RAFI IMAM NASRULLAH
109082530010
S1 IF-12-02**

**DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng**

**PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026**

SOAL LATIHAN

1. SOALA

Source Code:

```
package main

import "fmt"

func main() {

    var x float64
    var jumlah float64
    var rata float64
    var n int

    fmt.Println("Masukkan bilangan (akhiri dengan 9999):")

    fmt.Scan(&x)

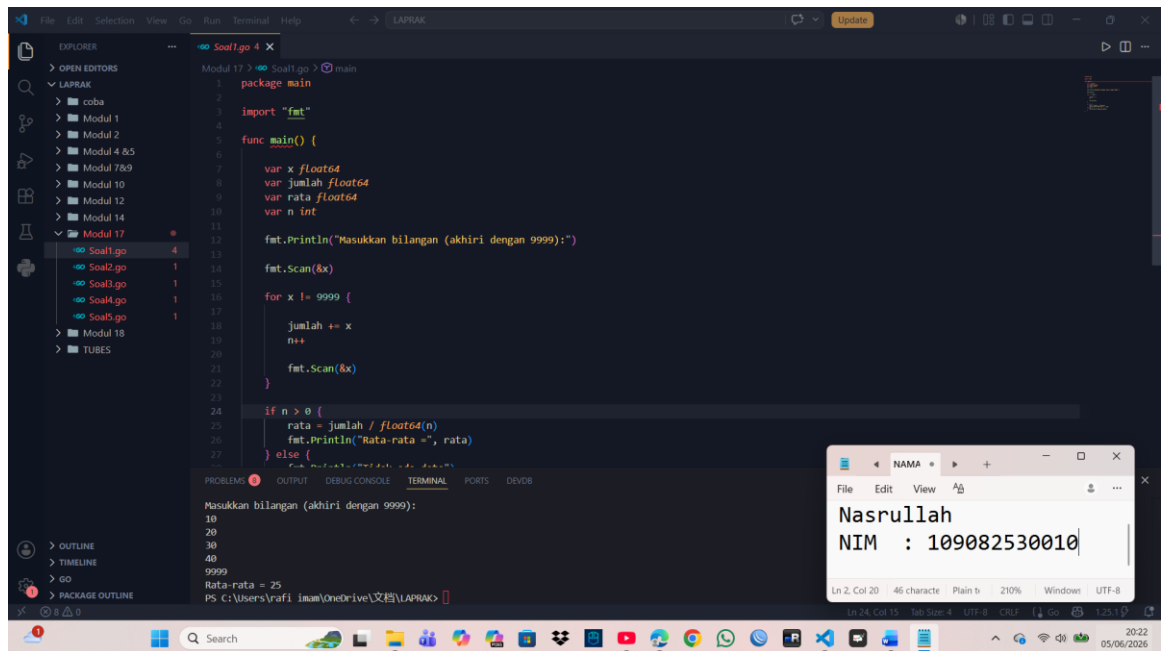
    for x != 9999 {

        jumlah += x
        n++

        fmt.Scan(&x)
    }

    if n > 0 {
        rata = jumlah / float64(n)
        fmt.Println("Rata-rata =", rata)
    } else {
        fmt.Println("Tidak ada data")
    }
}
```

Output:



Deskripsi:

Program ini digunakan untuk menghitung rata-rata dari sejumlah bilangan real yang dimasukkan oleh pengguna. Pengguna diminta memasukkan bilangan satu per satu, kemudian program akan terus membaca dan menjumlahkan setiap bilangan yang dimasukkan selama nilainya bukan 9999. Angka 9999 berfungsi sebagai penanda (marker) akhir input sehingga tidak ikut dihitung sebagai data. Setiap bilangan yang valid akan ditambahkan ke variabel jumlah, sementara variabel n digunakan untuk menghitung banyaknya data yang telah dimasukkan. Setelah pengguna memasukkan angka 9999, program akan menghentikan proses input dan memeriksa apakah ada data yang telah dimasukkan. Jika terdapat data, program menghitung rata-rata dengan membagi jumlah seluruh bilangan dengan banyaknya data, kemudian menampilkan hasil rata-rata tersebut. Namun, jika pengguna langsung memasukkan 9999 tanpa memasukkan data apa pun, program akan menampilkan pesan "Tidak ada data" karena tidak ada bilangan yang dapat dihitung rata-ratanya. Program ini menerapkan konsep perulangan dengan sentinel atau marker untuk menentukan akhir proses pembacaan data.

2. SOALA

Source Code:

```
package main

import "fmt"

func main() {

    var x string
    var n int

    fmt.Print("Masukkan string x: ")
    fmt.Scan(&x)

    fmt.Print("Jumlah data string: ")
    fmt.Scan(&n)

    data := make([]string, n)

    fmt.Println("Masukkan data string:")

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    ketemu := false

    posisi := -1
    jumlah := 0

    for i := 0; i < n; i++ {

        if data[i] == x {

            jumlah++

            if !ketemu {
                ketemu = true
                posisi = i + 1
            }
        }
    }

    if ketemu {
        fmt.Println("String ditemukan")
    } else {
        fmt.Println("String tidak ditemukan")
    }

    if posisi != -1 {
```

```

        fmt.Println("Posisi pertama:", posisi)
    }
    fmt.Println("Jumlah string x:", jumlah)
    if jumlah >= 2 {
        fmt.Println("Terdapat minimal dua string x")
    } else {
        fmt.Println("Tidak sampai dua")
    }
}

```

Output:

```

package main

import "fmt"

func main() {
    var x string
    var n int

    fmt.Print("Masukkan string x: ")
    fmt.Scan(&x)

    fmt.Print("Jumlah data string: ")
    fmt.Scan(&n)

    data := make([]string, n)

    fmt.Println("Masukkan data string:")

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    ketemu := false

    posisi := -1

    jeruk
    apel
    pisang
    String ditemukan
    Posisi pertama: 2
    Jumlah string x: 2
    Terdapat minimal dua string x
    PS C:\Users\rafi imam\OneDrive\文档\LAPRAK>

```

Deskripsi :

Program ini digunakan untuk mencari dan menganalisis kemunculan suatu string dalam sekumpulan data string yang dimasukkan oleh pengguna. Pada awal program, pengguna diminta memasukkan sebuah string x yang akan dicari, kemudian memasukkan jumlah data string (n). Setelah itu, pengguna menginput sebanyak n buah string yang disimpan ke dalam sebuah slice bernama data. Program kemudian melakukan pemeriksaan terhadap setiap elemen dalam slice tersebut untuk mengetahui apakah string x terdapat di dalam kumpulan data. Jika ditemukan, variabel ketemu akan bernilai true, jumlah kemunculan string x akan dihitung menggunakan variabel jumlah, dan posisi kemunculan pertama akan disimpan dalam variabel posisi. Setelah proses pencarian selesai, program menampilkan informasi apakah string x ditemukan atau tidak, posisi pertama string tersebut ditemukan, jumlah total kemunculannya dalam data, serta menentukan apakah string x muncul sedikitnya dua kali. Dengan demikian,

program ini dapat menjawab seluruh pertanyaan pada soal, yaitu keberadaan string x, posisi pertama kemunculannya, jumlah kemunculannya, dan apakah kemunculannya mencapai minimal dua kali dalam kumpulan data yang diberikan.

3. SOALA

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

func main() {

    var n int

    fmt.Print("Jumlah tetesan hujan: ")
    fmt.Scan(&n)

    rand.Seed(time.Now().UnixNano())

    var A, B, C, D int

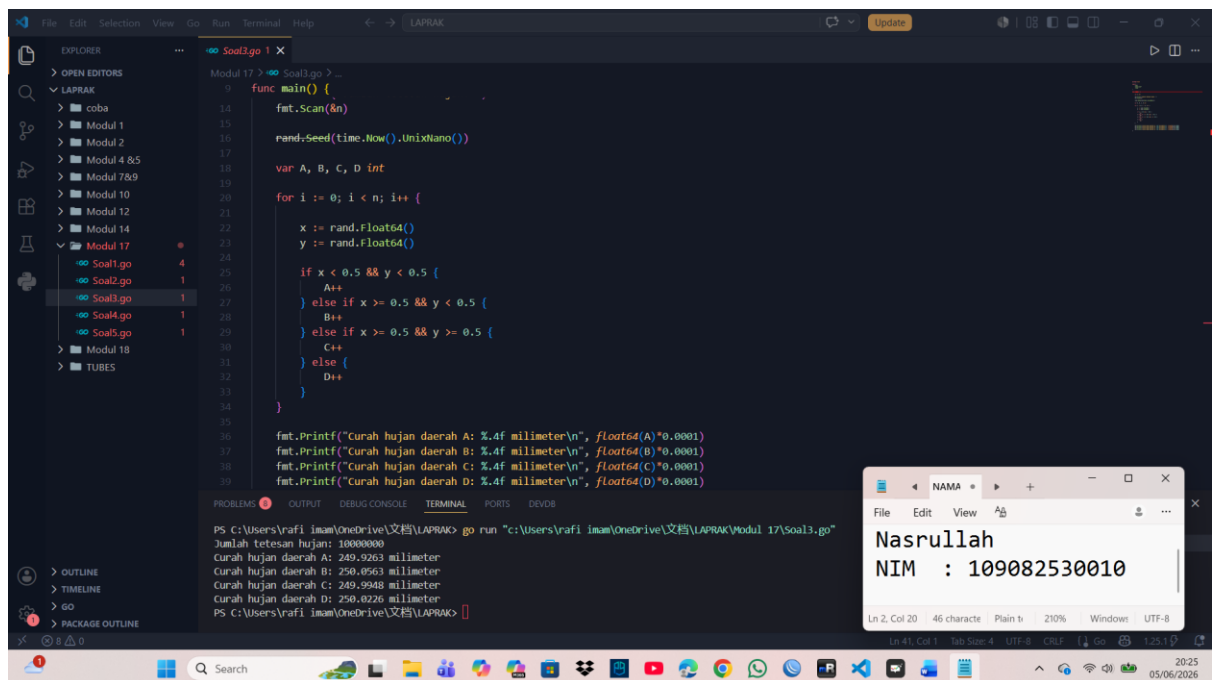
    for i := 0; i < n; i++ {

        x := rand.Float64()
        y := rand.Float64()

        if x < 0.5 && y < 0.5 {
            A++
        } else if x >= 0.5 && y < 0.5 {
            B++
        } else if x >= 0.5 && y >= 0.5 {
            C++
        } else {
            D++
        }
    }

    fmt.Printf("Curah hujan daerah A: %.4f milimeter\n", float64(A)*0.0001)
    fmt.Printf("Curah hujan daerah B: %.4f milimeter\n", float64(B)*0.0001)
    fmt.Printf("Curah hujan daerah C: %.4f milimeter\n", float64(C)*0.0001)
    fmt.Printf("Curah hujan daerah D: %.4f milimeter\n", float64(D)*0.0001)
}
```

Output:



```
func main() {
    fmt.Scan(&n)

    rand.Seed(time.Now().UnixNano())

    var A, B, C, D int

    for i := 0; i < n; i++ {

        x := rand.Float64()
        y := rand.Float64()

        if x < 0.5 && y < 0.5 {
            A++
        } else if x >= 0.5 && y < 0.5 {
            B++
        } else if x >= 0.5 && y >= 0.5 {
            C++
        } else {
            D++
        }
    }

    fmt.Printf("Curah hujan daerah A: %.4f milimeter\n", float64(A)*0.0001)
    fmt.Printf("Curah hujan daerah B: %.4f milimeter\n", float64(B)*0.0001)
    fmt.Printf("Curah hujan daerah C: %.4f milimeter\n", float64(C)*0.0001)
    fmt.Printf("Curah hujan daerah D: %.4f milimeter\n", float64(D)*0.0001)
}
```

```
PS C:\Users\rafi imam\OneDrive\文档\LAPRAK> go run "c:\Users\rafi imam\OneDrive\文档\LAPRAK\Modul 17\Soal3.go"
Jumlah tetesan hujan: 1000000
Curah hujan daerah A: 249.9263 milimeter
Curah hujan daerah B: 250.0563 milimeter
Curah hujan daerah C: 249.9948 milimeter
Curah hujan daerah D: 250.0226 milimeter
PS C:\Users\rafi imam\OneDrive\文档\LAPRAK>
```

deskripsi :

Program ini mensimulasikan curah hujan pada empat daerah, yaitu A, B, C, dan D. Pengguna memasukkan jumlah tetesan hujan yang akan disimulasikan, kemudian program menghasilkan koordinat acak (x, y) untuk setiap tetesan. Berdasarkan posisi koordinat tersebut, tetesan hujan akan dihitung ke salah satu dari empat daerah. Setelah seluruh tetesan diproses, program menghitung dan menampilkan curah hujan pada masing-masing daerah dalam satuan milimeter, dengan asumsi setiap tetesan hujan setara dengan 0,0001 milimeter curah hujan.

4. SOALA

Source Code:

```
package main

import "fmt"

func main() {

    var jumlah float64
    var si, sip1 float64
    var i int

    for i = 0; i++ {

        if i%2 == 0 {
            si = 1.0 / float64(2*i+1)
```

```

    } else {
        si = -1.0 / float64(2*i+1)
    }

    jumlah += si

    if (i+1)%2 == 0 {
        sip1 = 1.0 / float64(2*(i+1)+1)
    } else {
        sip1 = -1.0 / float64(2*(i+1)+1)
    }

    selisih := si - sip1

    if selisih < 0 {
        selisih = -selisih
    }

    if selisih <= 0.00001 {

        fmt.Println("Hasil PI :", 4*jumlah)
        fmt.Println("Pada i ke:", i+1)

        break
    }
}
}
}

```

Output:

The screenshot shows a Go IDE with the following code in `Soal4.go`:

```

package main

import "fmt"

func main() {
    var jumlah float64
    var si, sip1 float64
    var i int

    for i = 0; i++ {
        if i%2 == 0 {
            si = 1.0 / float64(2*i+1)
        } else {
            si = -1.0 / float64(2*i+1)
        }

        jumlah += si

        if (i+1)%2 == 0 {
            sip1 = 1.0 / float64(2*(i+1)+1)
        } else {
            sip1 = -1.0 / float64(2*(i+1)+1)
        }

        selisih := si - sip1
    }

    fmt.Println("Hasil PI :", 4*jumlah)
    fmt.Println("Pada i ke:", i+1)
}

```

The terminal output shows the execution of the program:

```

PS C:\Users\rafi_imam\OneDrive\文档\LAPRAK> go run "c:\Users\rafi_imam\OneDrive\文档\LAPRAK\Modul 17\Soal4.go"
Hasil PI : 3.1416026534897203
Pada i ke: 100001
PS C:\Users\rafi_imam\OneDrive\文档\LAPRAK>

```

A small window titled "NAMA" is also visible, displaying the user's name and NIM:

```

NAMA : Rafi Imam Nasrullah
NIM  : 109082530010

```


Deskripsi:

Program ini digunakan untuk menghitung nilai pendekatan π (pi) menggunakan metode deret tak hingga (seri Leibniz). Program menjumlahkan suku-suku secara berulang dengan tanda bergantian positif dan negatif, di mana setiap suku berbentuk pecahan $1/(2i+1)$. Proses perhitungan terus dilakukan sampai selisih antara dua suku berturut-turut menjadi sangat kecil, yaitu kurang dari atau sama dengan 0,00001. Ketika kondisi tersebut terpenuhi, program akan berhenti dan menampilkan hasil pendekatan nilai π (yaitu 4 kali jumlah deret) serta jumlah iterasi yang diperlukan untuk mencapai tingkat ketelitian tersebut.

5. SOALA

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n int
    fmt.Print("Banyak Topping: ")
    fmt.Scan(&n)

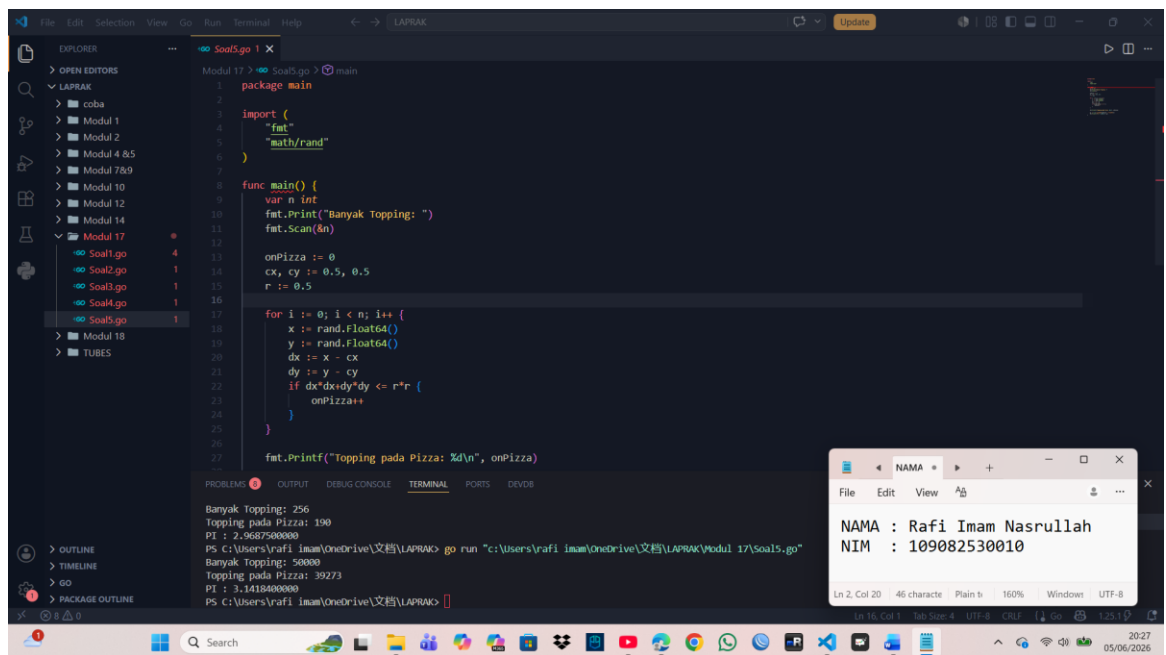
    onPizza := 0
    cx, cy := 0.5, 0.5
    r := 0.5

    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()
        dx := x - cx
        dy := y - cy
        if dx*dx+dy*dy <= r*r {
            onPizza++
        }
    }

    fmt.Printf("Topping pada Pizza: %d\n", onPizza)

    pi := 4.0 * float64(onPizza) / float64(n)
    fmt.Printf("PI : %.10f\n", pi)
}
```

Output:



The screenshot shows a Go IDE with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code in the editor is a Go program that simulates a pizza topping distribution. It uses a loop to generate random points (x, y) and checks if they fall within a circle of radius 0.5 centered at (0.5, 0.5). The terminal output shows the results of the simulation for 256, 5000, and 39273 toppings, along with the calculated value of pi.

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n int
    fmt.Print("Banyak Topping: ")
    fmt.Scan(&n)

    onPizza := 0
    cx, cy := 0.5, 0.5
    r := 0.5

    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()
        dx := x - cx
        dy := y - cy
        if dx*dx+dy*dy <= r*r {
            onPizza++
        }
    }

    fmt.Printf("Topping pada Pizza: %d\n", onPizza)
}
```

Terminal Output:

```
Banyak Topping: 256
Topping pada Pizza: 190
PI : 2.9687500000
PS C:\Users\rafi_imam\OneDrive\文档\LAPRAK> go run "c:\Users\rafi_imam\OneDrive\文档\LAPRAK\Modul 17\Soal5.go"
Banyak Topping: 5000
Topping pada Pizza: 39273
PI : 3.1418400000
PS C:\Users\rafi_imam\OneDrive\文档\LAPRAK>
```

Deskripsi :

Program ini digunakan untuk melakukan simulasi dalam menghitung nilai pendekatan π (pi) menggunakan metode Monte Carlo. Pengguna memasukkan jumlah percobaan berupa banyaknya “topping” yang disimulasikan. Setiap titik acak (x, y) dihasilkan di dalam bidang persegi, kemudian dicek apakah titik tersebut berada di dalam lingkaran dengan pusat (0.5, 0.5) dan jari-jari 0.5. Jika titik berada di dalam lingkaran, maka dihitung sebagai “topping pada pizza”. Setelah semua titik diproses, program menampilkan jumlah titik yang berada di dalam lingkaran serta menghitung pendekatan nilai π berdasarkan perbandingan jumlah titik di dalam lingkaran terhadap total titik yang dihasilkan.